Design a QR code Generator

QR code generator 是一种工具（software、library 或 service），用来接收输入数据（例如 URL、email、或 unique ID），并将其转换成 QR code 图像——一种由黑白方块组成的二维（2-D）矩阵，可被相机或 App 扫描。

功能性需求（Functional Requirements）

- 用户可以上传指定网址（ASCII，最长 20 字符），服务会回传产生的 QR code。
- 用户可以管理自己创建过的 QR Code。
- 当用户扫描 QR code 时，会被重新导向（redirect）到原始的 URL。

非功能性需求（Non-Functional Requirements）

- High Availability：服务需 24/7 可用。
- Redirection 延迟需极低（< 100ms）。
- 支持 10 亿个 QR codes 与 1 亿用户。

API 设计（API design）

```
API
# Create a QR Code
POST v1/qr_code
{
  url
}
Response: {
  qr_token
}
```

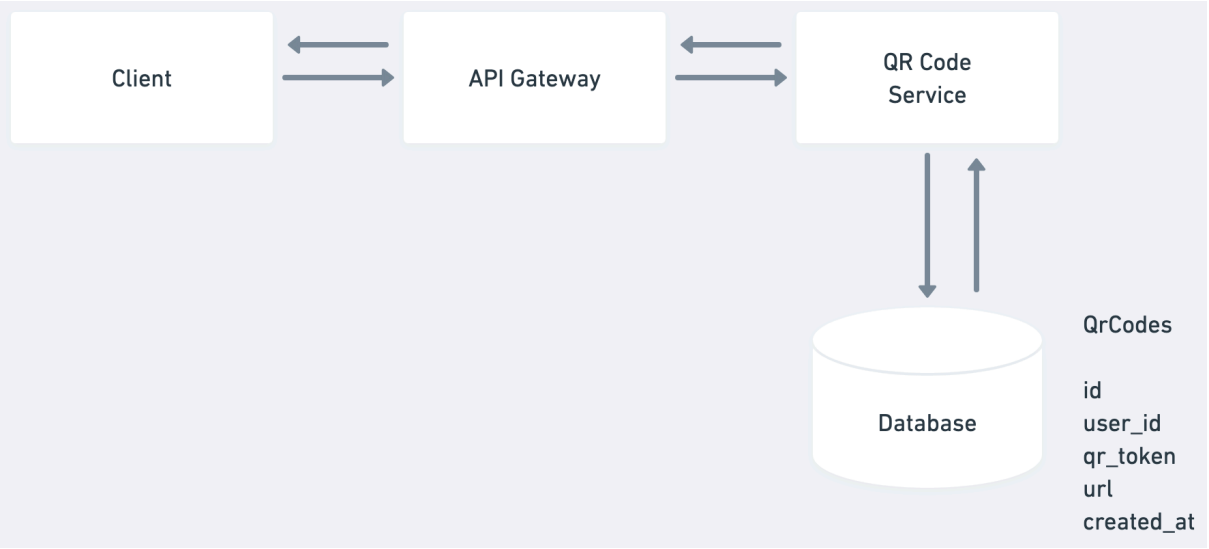
buildmoat.org

```
# Get QR Code image
GET v1/qr_code_image/:qr_token?dimension={dimension}&color=
{color}&border={border}

Response: {
  image_location
}

# Edit a QR Code
PUT v1/qr_code/:qr_token {
    url
}

# Delete a QR Code
DELETE v1/qr_code/:qr_token

# Get the original url
GET v1/qr_code/:qr_token
Response: {
   url
}
```

buildmoat.org

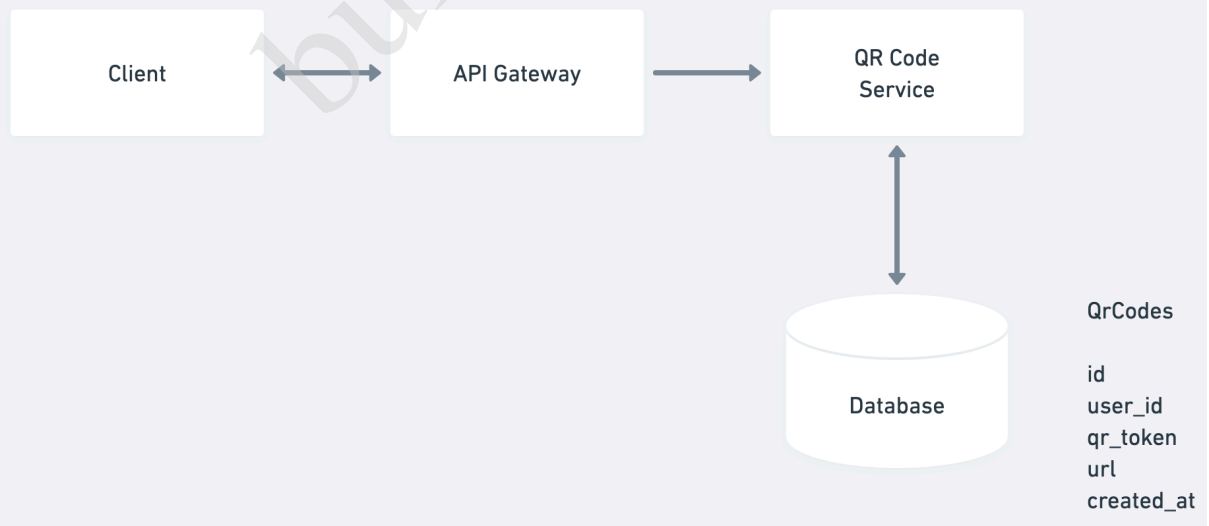
High-Level Design

1. QR Code Creation / Edit Flow



- 用户调用 POST `v1/qr_code`，并在 request body 中带入 url。
- 服务先验证 URL 是否合法。若合法，则产生一个全域唯一的 token：`qr_token`，并在 `QrCodes` table 中创建一笔数据。
  - 暂时假设我们有一个 black box 可以产生 unique token，细节会在 deep dive 说明。
- `qr_token` 在所有用户之间必须全域唯一（由 DB schema 保证）。
- Server 回传包含 `qr_token` 的 response。

2. QR Code Retrieval Flow



- 用户调用 GET `v1/qr_code_image/:qr_token?dimension=300&color=000000&border=10` 通过 query parameters 指定 image spec。

buildmoat.org

- Server 依照 token 产生 QR code 图像，然后回传 image resource location。
- QR Code 内嵌的 URL 会是 `https://myqrcode.com/qr_token`。当用户连接时，backend 会查找 `QrCodes` table，并回传 HTTP redirect 到原始 URL。常见有两种 redirect：
  - 301 (Permanent Redirect)：浏览器会缓存结果，未来可能直接跳过我们的 server。
  - 302 (Temporary Redirect)：浏览器不会缓存，确保每次都经过我们的 server。
  - 这里选择 302，因为我们希望 QR code 拥有者可以删除或修改对应关系，确保每次都取得最新状态。
- 用户也可调用 GET `v1/qr_code/:qr_token` 直接取得原始 URL（backend 同样查找 `QrCodes` table）。

深入探讨（Deep Dives）

1. 如何产生唯一 token

我们需要足够的熵（entropy）来让产生的 token 尽可能保持唯一性。为此，可以使用像 SHA-256 这样的哈希函数来产生固定长度的 hash 值。哈希函数会把输入转成一个确定性（deterministic）、固定长度的字符串。纯哈希函数是确定性的：同一个长网址永远会对应到同一个短码，而不需要查数据库。

如果想避免这种「同输入必得同输出」的情况，可以在输入中加入一个 secret 或 nonce（在密码学中指只使用一次、任意或不重复的随机数值），让每次产生的结果都不同。

完成 hashing 之后，可以把输出再用编码方式（例如 Base62）转成文本，让二进位数据变成更短、更容易传输的字符串。接着取前 N 个字符当作精简后的唯一 token。N 的大小取决于 key space 的大小，用来把 collision（碰撞）几率降到可接受的程度。

Base62 说明

Base62 使用 62 个字符 `[0-9A-Za-z]` 来表示数字，例如把 3842 转成 Base62：

3842 ÷ 62 = 61 余 60 → “y”

61 ÷ 62 = 0 余 61 → “z”

反向读取余数 → `[61, 60]` 对照 Base62 字符表 → `zy`

buildboat.org

即使如此，当 key space 变得够大时，仍然可能发生 collision。为了解决这个问题，可以在数据库中把 `qr_token` 设为 UNIQUE。当发生碰撞时，数据库会回报错误，我们再重新产生一个 token 并重试即可。

2. 如何确保 redirect 速度快

Indexing（索引）

为了避免 full table scan（全表扫描），我们可以在数据库中为 `qr_token` 创建 index。它可以是 primary index，也可以是 secondary index。要注意的是，如果把它设为 primary index，就同时会具有 UNIQUE 的特性。

Index 可以想像成书本的目录或图书馆的卡片目录，它让我们不用一页一页翻、一本一本找，就能快速定位到需要的数据。

Caching（缓存）

我们的服务是 read-heavy（读多写少，write:read ≈ 1:100）。如果每一次读取都直接打到数据库，当流量上来时，DB 的性能很快就会成为瓶颈。

举例来说：

100,000,000 users × 5 redirects = 500,000,000 redirects / day

500,000,000 ÷ 86,400 seconds ≈ 5,787 redirects / second

其中一个作法是使用 cache，把常被查找的结果先存在缓存中。每次读取时先查 cache，只有 cache miss 才打到 DB。写入时则同时更新 DB，并在必要时（例如删除）做 cache invalidation。

Cache 可以是：

- 每台 server instance 的 local cache（实作简单，但 hit rate 低）
- 独立的 distributed cache（例如 Redis / Memcached，hit rate 高，但系统复杂度与成本也较高）

事实上，大多数 DB 自身在访问磁盘前也有 buffer cache，只是弹性与功能不如独立的 cache layer 强。

CDN

CDN（Content Delivery Network）就是把网站或 APP 的静态内容（图片、视频、JS、CSS、QR 图等）预先拷贝到全球各地的 edge nodes，让用户从最近的节点取得数据，减少跨国或跨洲传输造成的延迟。

buildnoat.org

在这个设计中，我们可以把 QR Code 和 URL mapping cache 在 CDN 节点上。对于热门的 QR Code，URL redirection 甚至可以直接在 CDN 完成，完全不需要经过我们的 application server，大幅降低整体 latency。

在我们的 design 里，QR Code 本身就是非常适合放在 CDN 的 static data。这样一来，对于高流量的 QR Code，request 可以直接由 CDN 回传对应的内容，而不必回源到我们的 server。

3. Scaling the System

- Server 采用 stateless 设计，表示在回应完成后不会保留任何 request state，因此可以很容易通过增加 instance 来做 horizontal scaling。
- Database：
  - 假设有 10 亿（1B）笔数据，每笔 200 bytes，总量约为 200GB，对于可预见的未来，一台单一 DB instance 应该就足以承载。
  - 为了 fault tolerance（容错），可以加入多个 read replica，因为读取流量远高于写入。当 write replica 挂掉时，可以把其中一个 read replica 升级为新的 write replica。
- 我们也需要考虑定期清理长时间没有被点击的 URL，例如先通知用户这些链接即将被删除，然后用 cron jobs 定期扫描数据库并移除过期纪录。
- 由于 QR code image 是静态资源，如果图片请求的 QPS 很高，又不希望每次都回源到 object store 读取，可以导入 CDN（Content Delivery Network），让不同地区的用户都能以更低 latency 取得图片。

bullmoat.org